

Go Code Quality & Idioms Handbook

Standard Specification for Idiomatic Go, Error Semantics, Testing & Memory Efficiency

Go 1.18 - Go 1.22+ Standard

REVIEWER HANDBOOK PURPOSE

This handbook defines official code quality standards, language idioms, and reviewer evaluation criteria for Go codebases. It incorporates modern language specifications (Go 1.18 through Go 1.22+), providing clear distinctions between code smells and idiomatic Go constructs.

1. Error Handling & Propagation Mechanics

Go treats errors as explicit return values rather than exceptional control-flow interventions (such as try/catch exceptions). Clean error propagation requires strict adherence to wrapping conventions, clear boundary decisions, and modern multi-error aggregation.

- **Errors as Explicit Values:** Never use panic or recover for standard control flow or operational error handling. Reserve panic strictly for unrecoverable programmer errors during startup (e.g., malformed regexp compilation).
- **Contextual Error Wrapping (%w vs %v):** Annotate errors with actionable context using `fmt.Errorf("...: %w", err)` when the caller must inspect the underlying error chain using `errors.Is` or `errors.As`. Use `%v` when internal implementation details should be intentionally obscured from callers.
- **The 'Handle Once' Rule:** Make exactly *one* decision per error. Either log the error and degrade gracefully, OR wrap and return it up the call stack. **Never log and return the same error**, as this pollutes logs with duplicate, uncoordinated entries.
- **Multiple Errors Aggregation (Go 1.20+):** Use standard library `errors.Join(err1, err2)` to combine multiple operational errors into a single error value instead of importing third-party multierror packages.

× ANTI-PATTERN / SMELL

```
func Fetch(id string) (*User, error) {
    u, err := db.Query(id)
    if err != nil {
        log.Printf("db error: %v", err) // SMELL: Log &
        Return
        return nil, err
    }
    return u, nil
}
```

✓ IDIOMATIC PATTERN

```
func Fetch(id string) (*User, error) {
    u, err := db.Query(id)
    if err != nil {
        // IDIOMATIC: Wrap & return once
        return nil, fmt.Errorf("fetching user %s: %w", id, err)
    }
    return u, nil
}
```

2. Code Clarity & Readability Standards

Idiomatic Go favors flat, linear readability over deep nesting or clever single-line expressions. The 'line of sight' principle keeps happy-path execution aligned along the left margin.

- **Guard Clauses & Line of Sight:** Handle preconditions, invalid inputs, and error states immediately with early return guard clauses. Eliminate else blocks after error returns to keep primary business logic un-indented.
- **Function Granularity & Cyclomatic Complexity:** Keep functions short and single-purposed. Functions with high branch complexity should be decomposed into focused helper functions.

Go Code Quality & Idioms Handbook

Standard Specification for Idiomatic Go, Error Semantics, Testing & Memory Efficiency

- **Documentation Commentary Standards:** Every exported symbol (func, struct, interface, const) MUST have a full-sentence doc comment starting with the identifier's name. Comments should explain *why* a component exists and its operational constraints, not merely repeat what the code does. Go 1.18 - Go 1.22+ Standard

× ANTI-PATTERN / SMELL

```
func Process(u *User) error {
    if u != nil {
        if u.Active {
            return u.Save()
        } else {
            return errors.New("inactive")
        }
    } else {
        return errors.New("nil user")
    }
}
```

✓ IDIOMATIC PATTERN

```
// Process validates and persists an active user.
func Process(u *User) error {
    if u == nil {
        return errors.New("user is nil")
    }
    if !u.Active {
        return errors.New("user is inactive")
    }
    return u.Save()
}
```

3. Naming, Packages & Type Idioms

Naming in Go communicates scope, lifecycle, and domain intent. Package layout must reflect functional business domains rather than generic technical roles.

- **Identifier Scope & Length:** Variable name length MUST be proportional to its scope. Use single-letter or short abbreviations for narrow local scopes (s for server, i for index, ctx for context). Use descriptive names for broad or exported package symbols.
- **Package Stutter Elimination:** Avoid repeating the package name in exported identifiers. In package user, export Entity or Service, not UserEntity or UserService (which stutters as user.UserEntity).
- **Prohibition of Generic Packages:** Banish catch-all package names like util, common, helpers, or base. Group functionality by domain feature (e.g., auth, tenant, payment).
- **Getters & Initialisms:** Do NOT prefix getter methods with Get. Field owner becomes method Owner(). Initialisms must preserve consistent casing throughout (e.g., userID, httpURL, XMLAPI, not userId or XmlApi).
- **Modern Type Aliases (Go 1.18+ any):** Enforce the standard any keyword as an alias for empty interface constraints (interface{}).
- **Black-Box Testing Boundary (package foo_test):** Prefer package foo_test for unit tests to verify the package strictly through its public exported API, ensuring clean decoupling.

× ANTI-PATTERN / SMELL

```
package util // SMELL: Generic package name

type UserUtil struct{} // SMELL: Stutter

func (u *UserUtil) GetUserId(interface{}) {} // Get
prefix & interface{}
```

✓ IDIOMATIC PATTERN

```
package user // IDIOMATIC: Domain package

type Service struct{} // Clean type name

func (s *Service) UserID(input any) {} // No 'Get',
capital ID, uses 'any'
```

4. Data Structures, Memory & Value Semantics

Understanding Go memory allocation, slice headers, and method receivers prevents memory churn, race conditions, and unnecessary garbage collection overhead.

Go Code Quality & Idioms Handbook

Standard Specification for Idiomatic Go, Error Semantics, Testing & Memory Efficiency

- **Useful Zero Values:** Design structs so their zero-value is immediately functional without requiring explicit initialization functions (e.g., `sync.Mutex`, `bytes.Buffer`).
Go 1.18 - Go 1.22+ Standard

- **Slice Initialization Idioms:** Prefer nil slice declarations (`var s []string`) over empty slice literals (`s := []string{}`) when declaring uninitialized slices. Both have length 0, but nil slices avoid unnecessary heap allocations.
- **Preallocation Mechanics:** Supply capacity hints when instantiating slices (`make([]T, 0, cap)`) and maps (`make(map[K]V, hint)`) when target size is known, eliminating memory re-allocation and copy overhead.
- **Method Receiver Semantics:** Use pointer receivers (`*T`) when the method mutates struct state, contains a `sync.Mutex`, or receives large structs. Use value receivers (`T`) for small, immutable types. Maintain strict receiver type consistency across all methods on a type.

× ANTI-PATTERN / SMELL

```
// SMELL: Unallocated slice appending in loop
var list []Order
for _, id := range ids {
    list = append(list, fetch(id))
}
```

✓ IDIOMATIC PATTERN

```
// IDIOMATIC: Preallocated slice memory
list := make([]Order, 0, len(ids))
for _, id := range ids {
    list = append(list, fetch(id))
}
```

5. Testing & Verification Standards

Go testing relies on table-driven structures, clean test helper attribution, and modern test execution mechanics to build robust, maintainable test suites.

- **Table-Driven Test Anatomy:** Group test cases into an anonymous slice of structs containing input variables, expected output (want), and test labels. Iterate over test cases using `t.Run(tt.name, ...)`.
- **Go 1.22+ Loop Variable Capture Semantics:** In Go 1.22+, loop variables in for loops are re-allocated per iteration. **Do NOT** require `tt := tt` re-bindings prior to calling `t.Parallel()` inside subtests.
- **Test Helpers & Cleanup Registration:** Mark test utility setup functions with `t.Helper()` so test failure line numbers point to the caller. Register teardown hooks via `t.Cleanup(func())` rather than deferred functions inside setup handlers.
- **Assertion Idioms & Diffs:** Format failure checks as got before want (e.g., `if got != tt.want`). Use `google/go-cmp` (`cmp.Diff`) for struct comparisons. Use `t.Error` for non-fatal table failures to allow other test cases to complete, and reserve `t.Fatal` for critical setup failures.

× ANTI-PATTERN / SMELL

```
func TestParse(t *testing.T) {
    for _, tt := range tests {
        tt := tt // OBSOLETE in Go 1.22+
        t.Run(tt.name, func(t *testing.T) {
            t.Parallel()
            // ...
        })
    }
}
```

✓ IDIOMATIC PATTERN

```
func TestParse(t *testing.T) {
    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            t.Parallel() // Clean in Go 1.22+
            if got := Parse(tt.in); got != tt.want {
                t.Errorf("Parse(%q) = %v, want %v", tt.in, got, tt.want)
            }
        })
    }
}
```

6. Concurrency Safety & Resource Management

Concurrency errors (goroutine leaks, race conditions, deadlocks) are among the most severe production defects. Strict goroutine ownership and modern sync idioms are mandatory.

Go Code Quality & Idioms Handbook

Standard Specification for Idiomatic Go, Error Semantics, Testing & Memory Efficiency

• **Goroutine Ownership Discipline:** The calling frame must ALWAYS know how and when a spawned goroutine terminates. Keep domain and application logic synchronous; manage goroutines exclusively in top-level worker/port wrappers. Go 1.18 - Go 1.22+ Standard

• **Prohibition of Copying Synchronization Primitives:** Never pass `sync.Mutex`, `sync.WaitGroup`, or structs containing them by value. Always pass them via pointer receivers or references to prevent lock state duplication.

• **Modern Lazy Initialization (Go 1.21+ `sync.OnceValue`):** Use `sync.OnceValue` or `sync.OnceValues` for safe, concise lazy initialization instead of custom boolean flags protected by raw mutexes.

• **Channel Sizing & Buffer Rules:** Default to unbuffered channels (capacity 0) for strict handoff synchronization. Use buffered channels strictly for fixed worker pools or rate-limiting semaphores where capacity is deterministic.

• **Defer Mechanics & Loop Lifetime Hazards:** Execute `defer resource.Close()` immediately following allocation. Never call `defer` inside a long-running for loop, as deferred calls execute only when the surrounding function returns, leading to resource exhaustion.

× ANTI-PATTERN / SMELL

```
// SMELL: Custom mutex flag for once init
type Cache struct {
    mu sync.Mutex
    done bool
    val string
}
```

✓ IDIOMATIC PATTERN

```
// IDIOMATIC: Go 1.21+ sync.OnceValue
var getVal = sync.OnceValue(func() string {
    return loadExpensiveConfig()
})
```

7. Code Quality Reviewer Grading Matrix

Code reviewers should evaluate proposed Go code against this 10-point checklist:

Check Category	Review Check / Standard	Anti-Pattern / Smell	Idiomatic Remediation
1. Errors	Errors handled once; wrapped with %w for inspection.	Log and return error; raw un-wrapped strings.	Wrap once with <code>fmt.Errorf("...: %w", err)</code> .
2. Multi-Errors	Use Go 1.20+ <code>errors.Join</code> for multiple errors.	Custom third-party multierror packages.	Use <code>errors.Join(err1, err2)</code> .
3. Readability	Guard clauses keep happy path left-aligned.	Deeply nested if/else blocks.	Return early on error/precondition.
4. Naming	No stutter; no Get getters; full initialisms.	<code>user.UserEntity</code> , <code>GetOwner()</code> , <code>userId</code> .	<code>user.Entity</code> , <code>Owner()</code> , <code>userID</code> .
5. Type Aliases	Use any keyword for empty interfaces.	Legacy <code>interface{}</code> type constraint.	Replace with <code>any</code> (Go 1.18+).
6. Packages	Domain-focused packages; no <code>util</code> .	<code>util</code> , <code>common</code> , <code>helpers</code> packages.	Move logic to feature packages (e.g. <code>auth</code>).
7. Memory	Nil slice init; preallocate slices/maps.	<code>s := []string{}; append without cap.</code>	<code>var s []string; make([]T, 0, cap)</code> .
8. Testing	Table-driven tests; no obsolete <code>tt := tt</code> .	Redundant <code>tt := tt</code> in Go 1.22+ loops.	Direct <code>t.Run</code> subtests with <code>t.Parallel()</code> .
9. Lazy Init	Use Go 1.21+ <code>sync.OnceValue</code> .	Manual mutex + boolean flag init.	Wrap setup in <code>sync.OnceValue(fn)</code> .
10. Concurrency	Explicit goroutine lifecycle; no mutex copying.	Fire-and-forget <code>go func()</code> ; value mutex.	Caller controls goroutines; pointer receivers.